ITigers — текст защиты

Основная версия: около 4 минут при спокойном темпе с показом экрана. Слайды 1–8 — основной рассказ; дополнительные слайды — для вопросов. Репетиция с таймером обязательна: темп выступающего важнее оценки по словам.

Основная речь, 3–5 минут

1. Продукт · 0:00–0:20

Здравствуйте! Мы команда ITigers. Представляем Альфа.Поток — умный кредитный кабинет предпринимателя. Мы собрали работающий веб-прототип, в котором клиент и сотрудник банка проходят один связанный процесс: от выбора продукта до решения, документов и демонстрационной выдачи.

2. Потребность · 0:20–0:45

В основе решения — две задачи из кейса. Предпринимателю нужно понять, какие условия выбрать, что происходит с заявкой и что делать дальше. Сотруднику банка нужно видеть данные и историю вместе, чтобы принять обоснованное решение. Мы разобрали процесс и проверили сценарии на прототипе. Потребности пока остаются гипотезами: интервью с реальными клиентами мы не проводили и не выдаём их за результаты исследования.

3. Кабинет клиента · 0:45–1:15

Клиент выбирает оборотный кредит, финансирование оборудования или овердрафт. Реквизиты компании уже заполнены. При изменении суммы и срока он видит ориентировочный платёж и предварительную оценку модели. После отправки заявка сохраняется в базе. В карточке отображаются результат, история и конкретное следующее действие. Материалы для бизнеса меняются по отрасли компании и открываются полностью.

4. Разные результаты · 1:15–1:45

Мы предусмотрели не только одобрение и отказ. Если нужны сведения, сотрудник запрашивает конкретные показатели или документ. Клиент отвечает в той же заявке, и контекст сохраняется. Если исходная сумма не подходит, банк предлагает другие условия. Исходный запрос остаётся видимым, а клиент отдельно принимает новое предложение. Это позволяет показать полноценный кредитный процесс, а не один экран с баллом.

5. Архитектура и ML · 1:45–2:15

Интерфейс работает на Vue, процесс и API — на Django, риск оценивает отдельный сервис FastAPI с LightGBM. Код и веса ML-разработчика мы не меняли: все сопоставления и ограничения находятся в адаптере Django. Скоринг и кредитное решение разделены. Если модель недоступна, заявка остаётся у сотрудника, а система не придумывает оценку. У модели есть ограничения: синтетические данные и фиксированная ставка 22 процента. Её показатель «100 минус PD» не заявляется гарантией одобрения.

6. Проверка и границы · 2:15–2:40

Проект запускается одной командой Docker Compose и опубликован по HTTPS. GitHub Actions проверяет API, миграции, схему и реальный ML перед обновлением. Проверены роли, чужие документы, повторные действия и ошибки модели. ФНС и БКИ представлены синтетическими данными. Подпись и перевод денег смоделированы: это явно написано в интерфейсе и PDF. Для реального внедрения потребуются банковские интеграции, идентификация и проверка модели на реальных согласованных данных.

7. Живой показ · 2:40–3:40

[Открыть сайт, профиль «Пекарня Зерно», создать стандартную заявку.]

Вот параметры заявки и предварительная оценка. Отправляем — система проверяет данные и сохраняет результат. Заявка одобрена, теперь клиент видит конкретные условия и принимает их. Появляется PDF-договор. После согласия и демонстрационного подписания фиксируется выдача. Перезагрузка не теряет заявку и историю. Все эти действия происходят через API и базу, а не переключают заранее подготовленные картинки.

[Если времени мало, показать заранее открытую карточку без создания новой.]

8. Завершение · 3:40–4:00

Результат ITigers — единый кабинет для клиента и банка с работающими ветками процесса и прозрачными действиями на каждом этапе. Код, Swagger, тестовые профили, диаграммы и сценарий проверки доступны в репозитории. Следующий шаг — интервью, реальные интеграции и валидация риск-модели. Спасибо! Готовы показать дополнительные сценарии и ответить на вопросы.

Короткая версия, около 3 минут

Здравствуйте! Мы ITigers, наш проект — Альфа.Поток, кредитный кабинет предпринимателя. Он связывает клиента и сотрудника банка в одном процессе: продукт, заявка, оценка риска, решение, документы и демонстрационная выдача.

Мы исходили из задач кейса: клиенту важно понимать условия и следующее действие, сотруднику — видеть данные, риск и историю вместе. Это рабочие гипотезы после разбора задания, а не подтверждённые интервью.

В кабинете три продукта. Компания заполнена заранее, платёж и предварительная оценка меняются вместе с параметрами. После отправки заявка сохраняется. Клиент видит понятное объяснение, сотрудник — внутренние показатели модели и сведения проверок. Отраслевые статьи меняются под компанию и открываются полностью.

Есть четыре результата рассмотрения: одобрение, отказ, запрос сведений и встречное предложение. При дозапросе клиент уточняет показатели и прикладывает документ в той же карточке. История не теряется. При встречных условиях исходная сумма сохраняется, а выдача использует именно принятый оффер.

Vue отвечает за интерфейс, Django — за процесс и кредитную политику, отдельный FastAPI с LightGBM — за риск. Код модели не менялся. Если она недоступна, система передаёт заявку

сотруднику. ФНС и БКИ синтетические, подпись и выдача смоделированы. Модель обучена на синтетике, поэтому промышленное качество мы не заявляем.

[Показать сайт в течение 45–60 секунд.]

Входим как пекарня, отправляем стандартную заявку. Получаем решение, принимаем условия, открываем договор и подтверждаем демоподписание. Фиксируется выдача. После обновления результат остаётся в базе.

Проект доступен по HTTPS, запускается одной командой Docker Compose, обновляется через GitHub Actions. В репозитории есть API, диаграммы, тестовые данные и сценарии проверки. Следующий шаг — проверка гипотез на интервью, реальные интеграции и банковская валидация модели. Спасибо!

Сценарий демонстрации и запасной вариант

Перед защитой открыть https://alpha.srv-node.com/ и подготовить второй таб с репозиторием. Нужен именно веб-браузер; мобильное приложение не требуется.

Основной показ: green → новая заявка → стандартные 2 млн / 24 месяца → отправить → принять → скачать PDF → согласие → демоподпись → DISBURSED. Не открывать админку на публичном экране и не демонстрировать пароли.

Дополнительно по вопросу жюри: yellow → REVIEW → сотрудник → дозапрос → PDF/уточнение → сотрудник → встречные 900 тыс. → клиент принимает. Для отказа — red, стандартная заявка. Для персонализации — «Новости» green/yellow.

Если внешний сервер или интернет недоступен: запустить локальный Compose заранее; URL http://127.0.0.1:5173/. Если запуск тоже невозможен, показать слайды с фактическими экранами и прямо сказать, что это резервные скриншоты. Не представлять их живой демонстрацией.

Ответы на вопросы

Где ИИ и зачем? LightGBM оценивает риск, даёт балл/PD и ориентир суммы. Ценность в предварительной обратной связи и поддержке сотрудника. Новости не генерируются LLM, распознавание документов не заявлено.

Вы меняли модель? Нет. Исходники, веса, обучение, requirements и Dockerfile в ml/ сохранены. Адаптер и фиксация окружения находятся за пределами ml/.

Как измерено качество? Проверена интеграция и ожидаемое поведение на синтетических сценариях. Качество на реальных заёмщиках и экономический эффект не измерены. Нужны отдельная выборка, AUC/PR-AUC, калибровка, стабильность по сегментам и согласованная кредитная политика.

100 − PD — вероятность одобрения? Нет. Это выход исходного сервиса, помеченный в интерфейсе как оценка 100 − PD. Решение Django учитывает стоп-факторы, балл, лимит, продукт и ручной анализ.

Почему модель считает по 22%, а продукт имеет другую ставку? Это ограничение неизменённого ML. Интерфейс и договор считают по ставке продукта/оффера. Для промышленного использования

разработчик модели должен согласовать контракт и политику; сейчас расхождение раскрыто, скрытой правки ML нет.

Что при сбое ML? Таймаут ограничен, заявка сохраняется в REVIEW с признаком недоступности. Сотрудник решает вручную. Поддельных показателей нет.

Что реально интегрировано? Vue ↔ Django ↔ исходный ML, SQLite, PDF, GitHub Actions и HTTPS работают. ФНС, БКИ, подпись и перевод — моки.

Почему SQLite? Для переносимого MVP с одной командой запуска и простой резервной копией. Для высокой нагрузки нужны PostgreSQL, очередь и отдельное хранилище документов. Промышленная масштабируемость не доказана.

Что с безопасностью? Проверяются роль и владелец, документы скачиваются через защищённый API, операции ограничены статусом, повторная подпись не создаёт вторую выдачу. Демо-вход не заменяет банковскую идентификацию.

Есть CustDev и измеренный эффект? Пока нет. Есть разбор кейса, гипотезы, связь потребностей с функциями и план проверки на пользователях. Цифры сокращения времени или роста конверсии не придуманы.

Что сдаём? Публичный репозиторий с историей, Docker/lock-файлы, Swagger, коллекция запросов, модель данных, диаграммы состояний и архитектуры, тестовые сценарии, презентация и текст защиты. Сдаваемую версию можно однозначно определить тегом и SHA; сам дедлайн организаторов команда проверяет.